

Vue Product Showcase: E-commerce ModaUno

Autor/a: Felipe Muñoz

Cuentas de prueba: cliente `cliente@modauno.cl / Demo1234` · administrador `admin@modauno.cl / Admin1234`.

1. Descripción de la aplicación

1.1 Funcionalidades principales

- **Catálogo** de 30 productos organizados en 4 categorías: Ropa Masculina, Ropa Femenina, Ropa Infantil y Calzado.
- **Filtros y rutas claras:** por categoría y por tipo de prenda (por ejemplo `/catalogo/ropa-infantil/vestidos`), búsqueda por nombre y orden por precio.
- **Detalle de producto** con cantidad, botón de agregar al carrito y botón de comprar ahora.
- **Cuentas de usuario simuladas:** registro e ingreso desde un modal, con cuentas de prueba (cliente y administrador) para facilitar la navegación.
- **Favoritos** (solo con sesión iniciada; cada usuario tiene su lista).
- **Carrito de compras** con cantidades, subtotal, envío y total en pesos chilenos.
- **Checkout con pago simulado** (con opción de simular un pago rechazado para probar errores), confirmación del pedido e historial en "Mis compras".
- **Panel de ventas** para el administrador, con indicadores y generación de ventas ficticias para pruebas.
- **Diseño responsive**, menú hamburguesa en pantallas pequeñas y **tema claro/oscuro**.

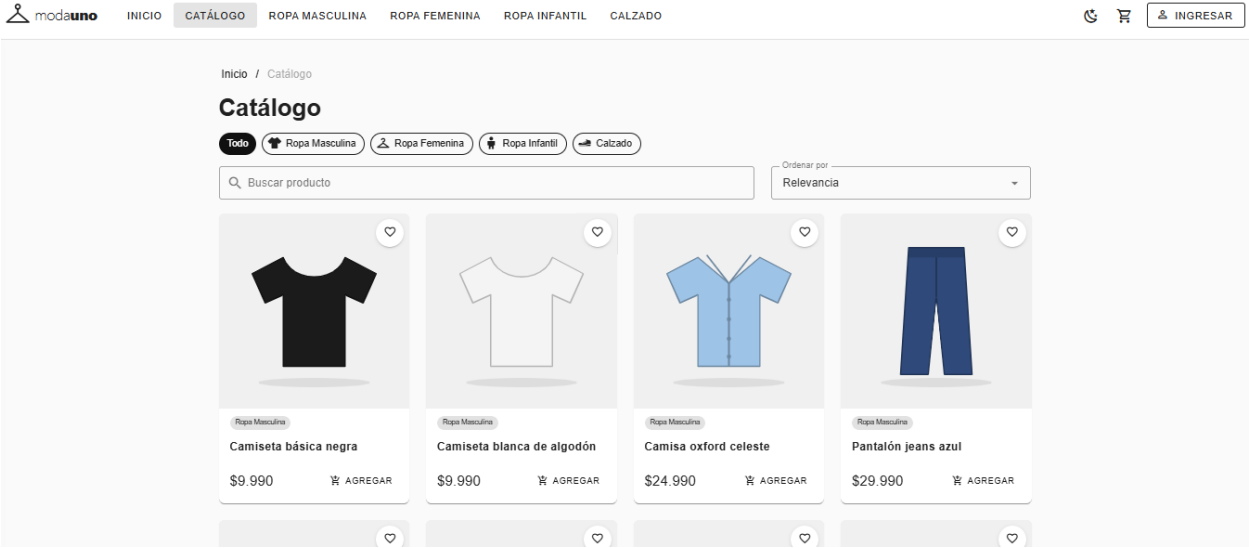
1.2 Mapa de rutas

Ruta	Página	Acceso
/	Inicio	Público
/catalogo, /catalogo/:categoria, /catalogo/:categoria/:tipo	Catálogo y subrutas	Público
/producto/:id	Detalle de producto	Público
/favoritos	Favoritos	Con sesión
/checkout	Datos de entrega y pago simulado	Con sesión

Ruta	Página	Acceso
/pedido/:id	Confirmación del pedido	Con sesión
/mis-compras	Historial de compras	Con sesión
/admin/ventas	Panel de ventas	Administrador



Captura 1: página de inicio en escritorio, con el bloque "Nueva moda".



Captura 2: catálogo con los filtros de categoría y tipo de prenda.

2. Tecnologías utilizadas y justificación

Tecnología	Para qué se usó	Justificación
Vue 3 + Vue CLI	Base del proyecto	Vue 3 permite componentes reutilizables y un ciclo de vida claro
Vue Router	Navegación entre páginas	Una SPA necesita rutas sin recargar la página; también protege las páginas privadas
Vuex	Estado global	Pedido explícito de la consigna; centraliza datos, filtros, favoritos, sesión, carrito y pedidos
Axios	Consumo de la API	Pedido de la consigna; maneja errores y tiempos de espera de forma simple
Vuetify	Librería de interfaz	Componentes accesibles, sistema de grillas responsive y tema claro/oscuro integrado
ESLint	Calidad del código	Mantiene un estilo consistente y bloquea malas prácticas (por ejemplo, la regla no-var)
Jest + Vue Test Utils	Pruebas unitarias	Herramientas pedidas por la consigna para probar componentes de forma aislada
Cypress	Pruebas end-to-end	Herramienta pedida por la consigna para probar flujos completos en el navegador

Las dependencias están fijadas en versiones exactas (con `package-lock.json`) para que el proyecto se instale igual en cualquier computador.

3. Justificación por lección

Lección 1: Componentes y ciclo de vida

Qué pedía la consigna: configurar el proyecto con Vue CLI, crear `<ProductCard>`, implementar al menos un ciclo de vida y diseñar la estructura base (`<App>`, `<Header>`, `<Footer>`, `<ProductList>`).

Qué se hizo:

- Proyecto configurado con Vue CLI.
- `ProductCard.vue` es un componente reusable y "tonto": recibe el producto por *props* y avisa por eventos (`toggle-favorite`, `add-to-cart`). Así se usa igual en el catálogo, el inicio y los favoritos.
- Ciclos de vida: `mounted` en `ProductCard` y `created` en `ProductList` y en las vistas (cargan el catálogo al entrar).
- Estructura base: `App.vue`, `AppHeader.vue`, `AppFooter.vue` y `ProductList.vue`, más componentes de apoyo (`CartDrawer`, `AuthDialog`, `OrderSummary`, `AppLogo`).

Por qué: separar en componentes pequeños hace el código más fácil de leer, probar y escalar.

Lección 2: Consumo de datos desde una API

Qué pedía la consigna: integrar Axios, obtener datos de una API (pública o mock), mostrarlos dinámicamente, gestionar estados `loading`, `error` y `empty`, y agregar un filtro por categoría.

Qué se hizo:

- Axios configurado en `src/services/api.js` y un servicio `productService.js` que pide `products.json`.
- `ProductList` muestra tres estados: cargando (esqueletos de tarjetas), error (mensaje con botón *Reintentar*) y vacío (mensaje cuando no hay resultados).
- Filtro por categoría y, además, por tipo de prenda, búsqueda y orden.

Decisión importante: API propia en lugar de una pública. Se evaluó primero la *Platzi Fake Store API*, pero al revisarla se encontró que:

- es pública y editable por cualquiera: la categoría de ropa había sido renombrada a *"Updated Category Name"*, lo que dejaba el catálogo sin ropa;
- mezcla productos reales con datos de prueba (títulos como `title-8acf...`, precios absurdos e imágenes rotas);
- sus datos están en inglés y en dólares;
- no tiene categoría de ropa infantil, que era un requisito del proyecto.

Por eso se creó una API REST simulada (`public/api/products.json`) con 30 productos en español, precios en pesos chilenos y las cuatro categorías pedidas, incluida **Ropa Infantil**. Axios la consume igual que a una API real y funciona sin servidor adicional, tanto en desarrollo como en GitHub Pages. Las imágenes son ilustraciones propias en SVG.

Lección 3: Almacenamiento de estado en Vuex

Qué pedía la consigna: configurar Vuex con módulos (productos, filtros, favoritos), mover el consumo de API a acciones, usar getters para los productos filtrados y conectar los componentes con el estado.

Qué se hizo:

Módulo	Responsabilidad
products	Acción <code>fetchProducts</code> (llama a la API), estados <code>loading</code> y <code>error</code> , getters <code>filteredProducts</code> , <code>availableTypes</code> y <code>productById</code>
filters	Categoría, tipo de prenda, búsqueda y orden
favorites	Lista de favoritos por usuario
auth	Sesión, roles y cuentas de prueba
cart	Productos, cantidades, subtotal, envío y total
orders	Pedidos, ventas simuladas e indicadores

Los componentes se conectan al estado con `mapState`, `mapGetters`, `mapActions` y `mapMutations`. Los módulos son *namespaced*, así cada uno queda aislado y la aplicación puede crecer sin mezclar responsabilidades.

Lección 4: Pruebas en Vue

Se desarrolla en detalle en las secciones 5 y 6.

Lección 5: Librerías y frameworks complementarios

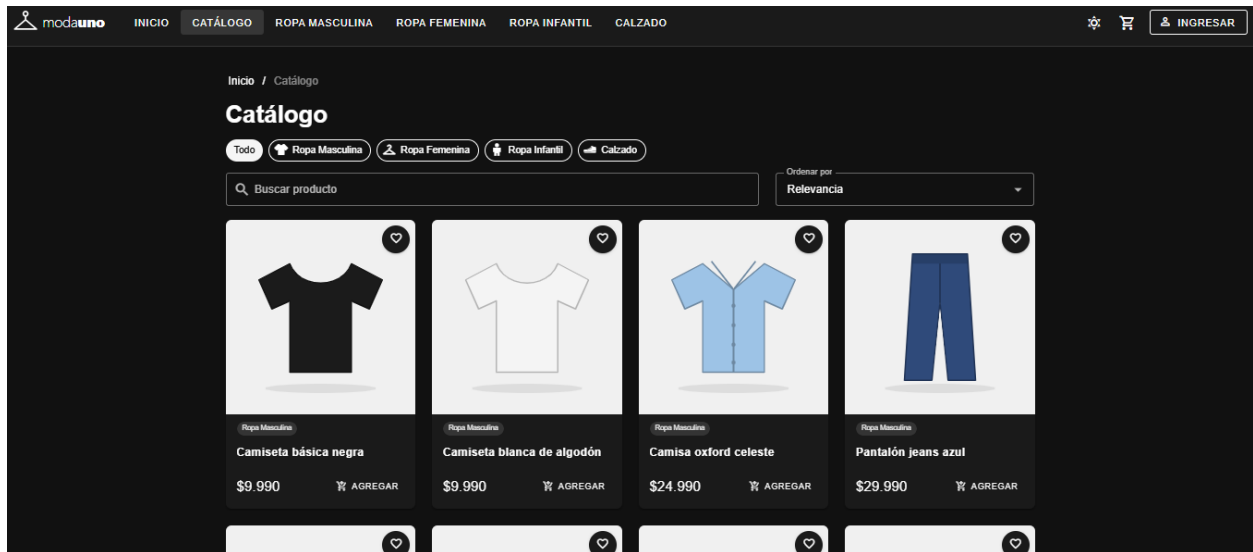
Qué pedía la consigna: elegir una librería UI, estilizar `<ProductCard>`, botones e inputs, (opcional) migrar a Nuxt o Quasar y justificar, y aplicar diseño responsive y tema claro/oscuro.

Qué se hizo:

- Se eligió **Vuetify** por su catálogo de componentes accesibles y su sistema de grillas.
- Tarjetas, botones, formularios, modal de ingreso, carrito lateral, tablas y paneles están contruidos con componentes de Vuetify.
- **Responsive:** la grilla pasa de 4 a 1 columna según el ancho; en pantallas menores a 1280 px el menú se transforma en menú hamburguesa.
- **Tema claro/oscuro** con botón en el navbar; la preferencia queda guardada en el navegador. El logo, en blanco y negro, se adapta a ambos temas.
- Diseño minimalista en blanco y negro, con animaciones suaves que se desactivan si el usuario prefiere menos movimiento.

4. Cumplimiento de los criterios de validación

Criterio de la consigna	Cómo se cumple	Evidencia
Arquitectura escalable por componentes	Componentes pequeños y reutilizables, vistas separadas, servicios y store por módulos	Estructura de carpetas en el repositorio
Integración y visualización correcta de datos	Axios + products.json, con estados de carga, error y vacío	Capturas 2 y 3
Gestión efectiva del estado global	Vuex con 6 módulos namespaced, acciones y getters	Pruebas de carrito y pedidos
Cobertura básica de pruebas	11 pruebas unitarias y 15 pruebas e2e	Secciones 6 y 7
Uso profesional de librerías UI	Vuetify en toda la interfaz, responsive y con tema claro/oscuro	Capturas 4 y 5
Entrega funcional, limpia y documentada	ESLint sin errores, build correcto, README y esta guía	Sección 8



Captura 3: tema oscuro.



Captura 4: vista móvil con el menú hamburguesa abierto.

5. Pruebas unitarias (Jest + Vue Test Utils)

5.1 Pruebas realizadas (5 archivos, 11 pruebas)

Archivo	Prueba	Qué comprueba
ProductCard.spec.js	Muestra título, categoría e imagen	Prueba 1 de la consigna: render correcto de <ProductCard>
ProductCard.spec.js	Emite toggle-favorite	Al tocar el corazón, el componente avisa con el id del producto
ProductCard.spec.js	Emite add-to-cart	Al presionar "Agregar", el componente avisa con el producto
ProductList.spec.js	Muestra un mensaje de error cuando la API falla	Prueba 2 de la consigna: se simula una falla de la API y se verifica que aparezca el mensaje "No pudimos cargar los productos" y ninguna tarjeta
cart.spec.js	Suma unidades del mismo producto	Agregar dos veces un producto deja una sola línea con la cantidad total
cart.spec.js	Calcula subtotal, envío y total	Con un producto de \$9.990 el envío es \$3.990; al superar \$50.000 el envío es gratis
cart.spec.js	Cambia cantidades y elimina	Una cantidad en cero elimina el producto

Archivo	Prueba	Qué comprueba
orders.spec.js	Crea un pedido pagado	Genera el pedido MU-0001 en estado "Pagado", con el total correcto, y vacía el carrito
orders.spec.js	Rechaza el pago simulado	Con la simulación de falla no se crea el pedido y el carrito se conserva
orders.spec.js	Numera pedidos y suma ventas	El segundo pedido es MU-0002 y las ventas totales suman ambos
productService.spec.js	Pide el catálogo y completa las rutas	Verifica la llamada a products.json y la ruta de las imágenes

5.2 Evidencia

```

modauno
├── cypress
├── docs
├── node_modules
├── public
├── scripts
├── src
├── tests
│   ├── unit
│   │   ├── esintrc.js
│   │   ├── gitignore
│   │   ├── npmrc
│   │   ├── babel.config.js
│   │   ├── cypress.config.js
│   │   ├── jest.config.js
│   │   ├── package-lock.json
│   │   ├── README.md
│   │   └── vue.config.js
└── ...

npm warn install-scripts
npm warn install-scripts Run 'npm install-scripts ls' to review, or 'npm install-scripts approve <pkg>' to allow.

C:\Users\fmunz\Desktop\modauno>npx jest --verbose
npm warn Unknown project config "cypress_install_binary". This will stop working in the next major version of npm. See 'npm help npmrc' for supported config options.
npm notice run modauno@2.0.0 npx
npm notice run jest --verbose
PASS tests/unit/cart.spec.js (0.627 s)
  carrito
    ✓ suma las unidades de un mismo producto (8 ms)
    ✓ calcula subtotal, envío y total (35 ms)
    ✓ cambia cantidades y elimina el producto al llegar a cero (2 ms)
PASS tests/unit/orders.spec.js (0.684 s)
  pedidos (ventas simuladas)
    ✓ crea un pedido pagado y vacía el carrito (30 ms)
    ✓ rechaza el pago cuando se simula una falla y conserva el carrito (34 ms)
    ✓ numera los pedidos en orden y suma las ventas totales (20 ms)
PASS tests/unit/productService.spec.js
  getProducts
    ✓ pide el catálogo y completa la ruta de las imágenes (3 ms)
PASS tests/unit/ProductList.spec.js (12.24 s)
  ProductList
    ✓ muestra un mensaje de error cuando la API falla (155 ms)
PASS tests/unit/ProductCard.spec.js (22.125 s)
  ProductCard
    ✓ muestra el título, la imagen y la categoría del producto (161 ms)
    ✓ emite "toggle-favorite" con el id al presionar el corazón (50 ms)
    ✓ emite "add-to-cart" con el producto al presionar Agregar (39 ms)

Test Suites: 5 passed, 5 total
Tests: 11 passed, 11 total
Snapshots: 0 total
Time: 41.523 s
Ran all test suites.

C:\Users\fmunz\Desktop\modauno>

```

Captura U1: terminal con el resultado completo de `npx jest --verbose` (los 5 archivos con PASS y la línea Tests: 11 passed).

6. Pruebas end-to-end (Cypress)

Se escribieron tres archivos de prueba (15 pruebas) en `cypress/e2e/`, y se buscan los elementos con atributos `data-test`, que no cambian aunque cambie el diseño.

Se ejecutan con la aplicación levantada (`npm run serve`) y el comando `npm run test:e2e:run`.

6.1 Prueba E2E 1: Flujo de login (`index.cy.js`)

Objetivo: comprobar que un usuario puede entrar, que el sistema rechaza una clave incorrecta y que se puede cerrar sesión.

Prueba	Pasos	Resultado esperado
Usuario puede iniciar sesión	Abre el modal desde el navbar, entra a la pestaña "Ingresar", escribe la cuenta demo y envía el formulario	Aparece el menú de la cuenta con el nombre "Cliente Demo"
Error con contraseña incorrecta	Escribe un correo válido con una clave equivocada	Se muestra el mensaje "Correo o contraseña incorrectos" y no hay sesión
Cerrar sesión	Entra y usa "Cerrar sesión" en el menú de la cuenta	Vuelve a aparecer el botón "Ingresar"

```
Running: index.cy.js (2 of 3)

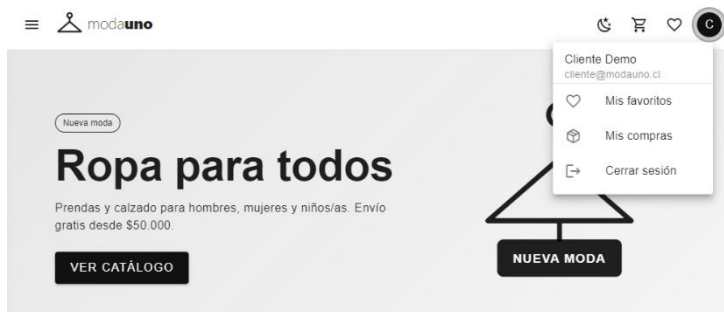
Flujo de Login
✓ Usuario puede iniciar sesión (11682ms)
✓ Muestra un error si la contraseña es incorrecta (10620ms)
✓ Usuario puede cerrar sesión (8732ms)

3 passing (31s)

(Results)

Tests: 3
Passing: 3
Failing: 0
Pending: 0
Skipped: 0
Screenshots: 2
Video: true
Duration: 31 seconds
Spec Ran: index.cy.js
```

Captura E1-a: resultado de `index.cy.js` en la terminal (3 pruebas con ✓).



Captura E1-b: login-exitoso.png

6.2 Prueba E2E 2: Catálogo (catalog.cy.js)

Objetivo: validar que el catálogo se muestra y que el usuario puede filtrar y encontrar productos. Incluye la **prueba E2E que pide la consigna**: "el usuario filtra productos y ve resultados".

Prueba	Pasos	Resultado esperado
Muestra los productos	Visita /catalogo	Hay más de 5 tarjetas de producto
Filtra por categoría (consigna)	Hace clic en el filtro "Calzado"	La URL pasa a /catalogo/calzado y todas las tarjetas son de Calzado
Filtra por tipo de prenda	Elige "Ropa Infantil" y luego "Vestidos"	La URL es /catalogo/ropa-infantil/vestidos y todas las tarjetas son vestidos
Busca por nombre	Escribe "botas" en el buscador	Queda solo "Botas de cuero café"
Sin resultados	Escribe "zzzz"	Aparece "No encontramos productos" y ninguna tarjeta
Abre el detalle	Hace clic en una tarjeta	Se abre /producto/. . . con el botón de agregar al carrito

```
Running: catalog.cy.js (1 of 3)

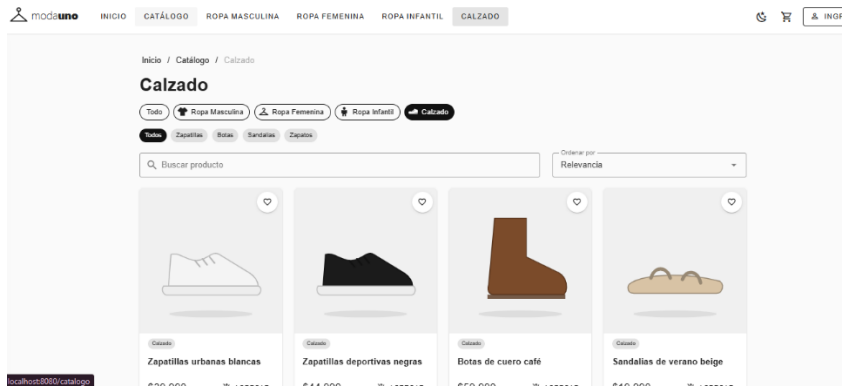
Catálogo
  ✓ muestra los productos (8517ms)
  ✓ el usuario filtra por Calzado y ve solo calzado (3363ms)
  ✓ filtra por tipo de prenda en una subruta (2917ms)
  ✓ busca un producto por nombre (2359ms)
  ✓ avisa cuando no hay resultados (3020ms)
  ✓ abre el detalle de un producto (6750ms)

6 passing (27s)

(Results)

Tests:      6
Passing:    6
Failing:    0
Pending:    0
Skipped:    0
Screenshots: 4
Video:      true
Duration:   27 seconds
Spec Ran:   catalog.cy.js
```

Captura E2-a: resultado de catalog.cy.js en la terminal (6 pruebas con ✓).



Captura E2-b: catalogo-calzado.png (el resultado del filtro por Calzado).

6.3 Prueba E2E 3: Compra simulada (purchase.cy.js)

Objetivo: validar el recorrido completo de una compra, que es el corazón de un e-commerce.

Prueba	Pasos	Resultado esperado
Agrega un producto al carrito	Presiona "Agregar" y abre el carrito	El producto aparece con su total en pesos
Cambia cantidad y elimina	Sube la cantidad a 2 y luego elimina el producto	La cantidad se actualiza y el carrito queda vacío
Pide iniciar sesión para pagar	Presiona "Ir a pagar" sin sesión	Se abre el modal con las cuentas de prueba
Completa la compra	Entra con la cuenta demo, llena los datos de entrega y paga	Se muestra el pedido MU-0001 en estado "Pagado"
Pago rechazado	Activa "Simular pago rechazado" y paga	Se muestra el error y se permanece en el checkout
Guarda la compra	Compra y visita /mis-compras	El pedido aparece en el historial

```
Running: purchase.cy.js (3 of 3)

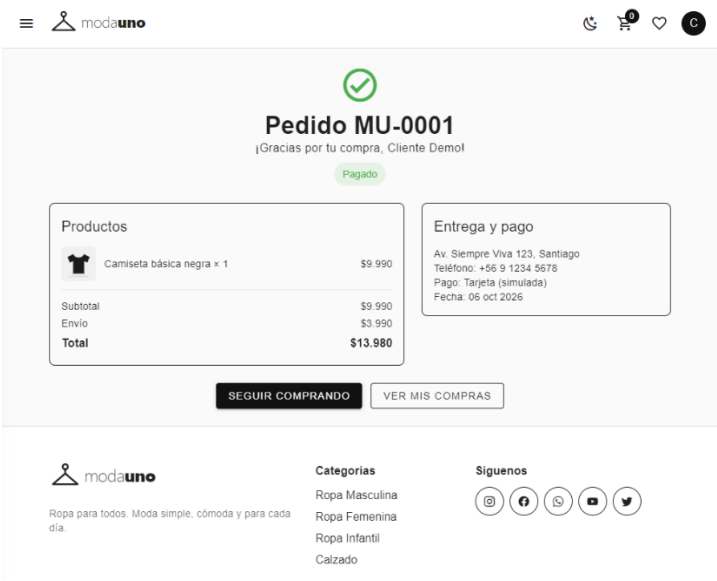
Compra simulada
✓ agrega un producto y lo ve en el carrito (9329ms)
✓ cambia la cantidad y elimina el producto (3850ms)
✓ pide iniciar sesión para pagar (4580ms)
✓ completa la compra y ve el pedido confirmado (14305ms)
✓ muestra un error si el pago es rechazado (10959ms)
✓ guarda la compra en "Mis compras" (12387ms)

6 passing (56s)

(Results)

Tests: 6
Passing: 6
Failing: 0
Pending: 0
Skipped: 0
Screenshots: 3
Video: true
Duration: 55 seconds
Spec Ran: purchase.cy.js
```

Captura E3-a: resultado de purchase.cy.js en la terminal (6 pruebas con ✔).



Captura E3-b: pedido-confirmado.png (pedido confirmado)

```
(Run Finished)

Spec                                Tests  Passing  Failing  Pending  Skipped
✓ catalog.cy.js                    00:27    6         6        -        -        -
✓ index.cy.js                      00:31    3         3        -        -        -
✓ purchase.cy.js                   00:55    6         6        -        -        -
✓ All specs passed!                01:54   15        15        -        -        -

C:\Users\fmunz\Desktop\modauno>
```

Captura: Resumen de pruebas realizadas

7. Capturas del producto final

modauno

INICIO

CATÁLOGO

ROPA MASCULINA

ROPA FEMENINA

ROPA INFANTIL

CALZADO

Finalizar compra

Datos de entrega

Nombre completo

Cliente Demo

Teléfono

Dirección

Comuna

Método de pago

☒ Tarjeta (simulada)

☐ Transferencia (simulada)

☐ Pago contra entrega (simulado)

Resumen

Camiseta básica negra × 1

\$9.990

Chaqueta bomber negra × 1

\$44.990

Subtotal

\$54.980

Envío

Gratis

Total

\$54.980

Captura 1: carrito abierto con productos y totales.

modauno

INICIO

CATÁLOGO

ROPA MASCULINA

ROPA FEMENINA

ROPA INFANTIL

CALZADO

✓

Pedido MU-0002

¡Gracias por tu compra, Cliente Demo!

Pagado

Productos

Camiseta básica negra × 1

\$9.990

Chaqueta bomber negra × 1

\$44.990

Subtotal

\$54.980

Envío

Gratis

Total

\$54.980

Entrega y pago

av. siempre viva 123, springfield

Teléfono: 123456

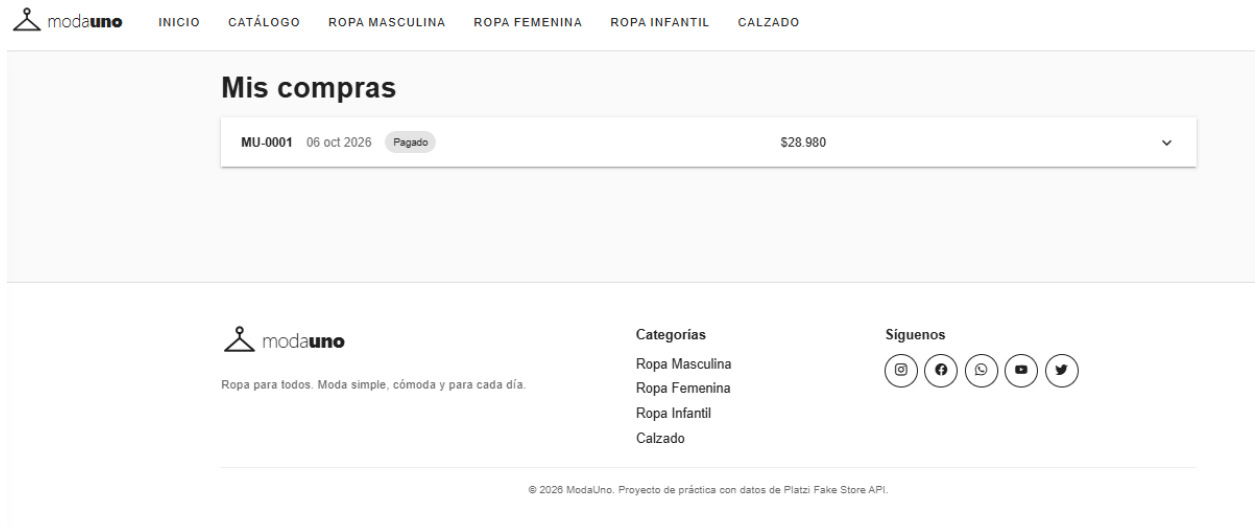
Pago: Tarjeta (simulada)

Fecha: 07 oct 2026

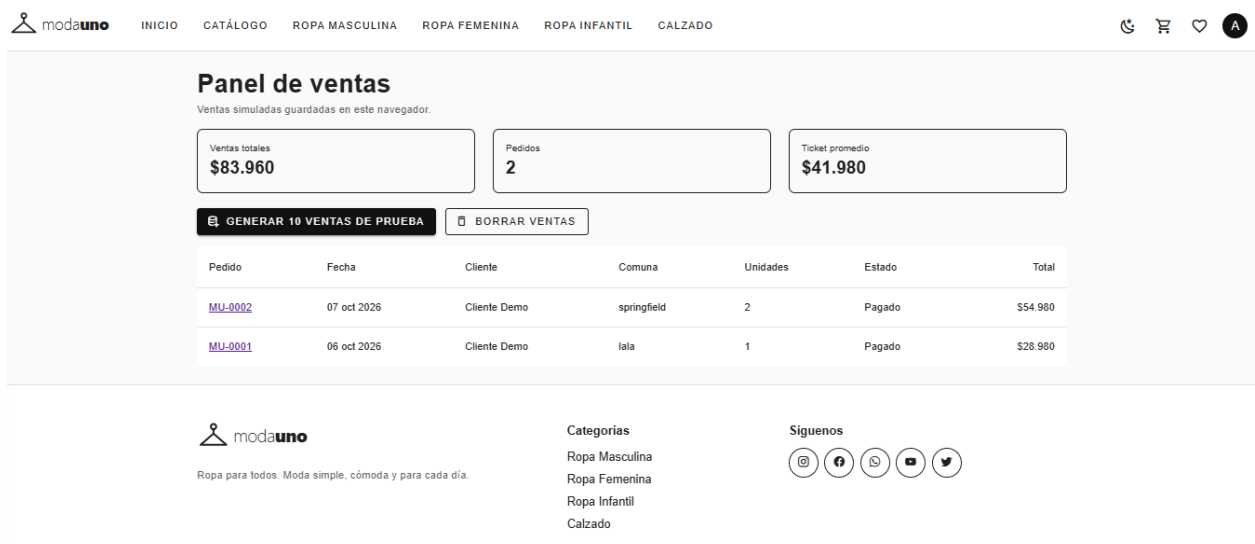
SEGUIR COMPRANDO

VER MIS COMPRAS

Captura 2: checkout con el resumen del pedido.



Captura 3: página "Mis compras".



Captura 4: panel de ventas (con la cuenta de administrador demo y ventas generadas).